1. **Baseline vs. Enhanced Prompts: Execute a naïve prompt ("Write a one-paragraph bio of Ada Lovelace.") and an enhanced prompt that adds role framing, specificity, and explicit format instructions; compare both outputs for relevance, completeness, and style.**

Baseline vs. Enhanced Prompt

A **prompt** is the instruction or question given to an AI model.

Baseline Prompt
A baseline prompt is a simple instruction with very little information.

Example :
Explain Artificial Intelligence.

Enhanced Prompt

An enhanced prompt includes additional information such as:

- Context
- Target audience
- Output format
- Length
- Examples
- Constraints

Example 1:

Explain Artificial Intelligence to first-year engineering students.

Requirements: 1. Use simple English. 2. Limit the answer to 200 words. 3. Give three real-life examples. 4. Mention advantages and disadvantages. 5. Present the answer using headings and bullet points.

(or)

Explain Artificial Intelligence to first-year engineering students using simple language. Include a definition, three real-life examples, advantages, and limitations. Present the answer using headings and bullet points.

2. **Iterative Refinement on a Simple Task: Summarize the plot of the Shakespearean play Romeo and Juliet in two sentences through three rounds of prompt tweaking:**
    a. **Minimal instruction.**
    b. **Addition of length and style constraints**
    c. **Specification of key content elements (setting and theme)**

   Document how each iteration changes and improves the result.

Ans:-

Iterative Prompt Refinement is the process of gradually improving a prompt by adding more context, instructions, and constraints. Each refinement helps the AI generate a more accurate and useful response.

Round 1: Minimal Instruction
Prompt:
Summarize the plot of the Shakespearean play Romeo and Juliet.

Observation

- General summary.
- No restriction on length or style.
- Provides the main idea but lacks specific formatting.

Round 2: Addition of Length and Style Constraints

Prompt:

Summarize the plot of the Shakespearean play Romeo and Juliet in exactly two sentences using simple English.

Round 3: Specification of Key Content Elements (Setting and Theme)

Prompt

Summarize the plot of the Shakespearean play Romeo and Juliet in exactly two sentences.

Requirements:

- Mention the setting (Verona, Italy).

- Mention the main characters (Romeo and Juliet).

- Highlight the central theme of love and family conflict.

- Use simple English.

Observation

- Includes the required setting.
- Mentions the main characters.
- Clearly highlights the central theme.
- Produces the most complete and structured response.

3. **Diagnosing Prompt Failures & Edge Cases: Craft a incomplete or contradictory prompt; analyze the failure mode (ambiguity, missing context, or format errors); refine the prompt by adding examples or clarifying instructions.**

   Aim
   To identify prompt failures caused by vague, contradictory, or incomplete instructions and improve the prompt by adding clear instructions and examples.

   Objective

- Understand common prompt failures.
- Identify ambiguity, missing context, and contradictory instructions.
- Refine prompts to obtain accurate and structured AI responses.

   Task 1: Incomplete Prompt
   Prompt
   Tell me about Python.

Failure Analysis

   Failure Type: Ambiguity (Missing Context)
   Reason:

- The prompt does not specify whether "Python" refers to the programming language or the snake.
- It does not specify the target audience or the required level of detail.

Refined Prompt

Explain the Python programming language to first-year engineering students.

Requirements:

- Use simple English.

- Limit the answer to 150 words.

- Mention three applications.

- Provide one simple Python code example.

Task 2: Contradictory Prompt

Prompt

Explain Artificial Intelligence in one sentence and provide a detailed explanation of at least 500 words.

Sample AI Response

The AI may:

- Ignore one of the instructions.
- Produce only one sentence.
- Produce a long explanation instead.

Failure Analysis

Failure Type: Contradictory Instructions

Reason:

- "One sentence" and "500 words" cannot both be satisfied.

Refined Prompt

Explain Artificial Intelligence in approximately 500 words.

Requirements:

- Use simple English.

- Include a definition.

- Explain three real-life applications.

- Mention advantages and limitations.

- Use headings and bullet points where appropriate.

Task 3: Format Error

Prompt

List the advantages of Artificial Intelligence.

Sample AI Response

The AI may return:

- A paragraph
- A numbered list
- A bulleted list

The format is unpredictable because no format is specified.

Failure Analysis

Failure Type: Format Error

Reason:

- The desired output format is not mentioned.

Refined Prompt

List five advantages of Artificial Intelligence in a table with two columns:

1. Advantage

2. Description

Exercise – 2

1. **Few-Shot vs. Zero-Shot Comparison: Design and execute a zero-shot prompt and a few-shot prompt (with 2–3 exemplar input-output pairs) for a chosen text task (e.g., sentiment classification or translation); compare outputs for accuracy, consistency, and adherence to examples.**

Few-shot prompt :
Few-shot prompting is a prompt engineering technique in which the user provides 2–3 (or a few) example input-output pairs before asking the AI to perform a similar task. These examples help the AI understand the expected format, style, and pattern of the response, leading to more accurate and consistent outputs.

Zero-Shot Prompting

In zero-shot prompting, the AI is given only the task without any examples.

Part A: Zero-Shot Prompt

The AI must classify each sentence as Positive, Negative, or Neutral.

Prompt

Classify the sentiment of the following sentence as Positive, Negative, or Neutral.

Sentence:

"The movie was entertaining and worth watching."

Observation

- Correctly identifies the sentiment.
- No examples are provided.
- The response format may vary depending on the AI model.

Part B: Few-Shot Prompt

Prompt

Classify the sentiment of each sentence as Positive, Negative, or Neutral.

Example 1:

Sentence: "I love this mobile phone."

Sentiment: Positive

Example 2:

Sentence: "The service was very poor."

Sentiment: Negative

Example 3:

Sentence: "The meeting starts at 10 AM."

Sentiment: Neutral

Now classify the following sentence:

Sentence: "The movie was entertaining and worth watching."

Observation

- Follows the same format as the examples.
- More consistent.
- Better understands the expected output style.

2. **Role-Based & Negative Prompting: Craft a role-based prompt to establish a specific persona (e.g., "You are a financial advisor…"); then create a negative prompt to suppress undesired content (e.g., "Do not mention any brand names"); evaluate how each influences the model's response.**

   Aim
   To understand how role-based prompting and negative prompting influence the responses generated by a Generative AI model.

   Role-Based Prompting
   Role-based prompting is a technique in which the AI is assigned a specific role or persona before answering a question. This helps the AI generate responses that match the expertise, tone, and perspective of that role.

Example Roles:

- Teacher
- Financial Advisor
- Doctor
- Software Engineer
- Career Counsellor

Negative Prompting

Negative prompting is a technique used to instruct the AI about what **not** to include in its response. It helps remove unwanted content and keeps the response focused on the user's requirements.

Examples:

- Do not mention any brand names.
- Do not use technical jargon.
- Do not exceed 150 words.
- Do not include code examples.

Example Of Role Based Prompting

You are a financial advisor.

Explain the importance of saving money to a college student.

Requirements:

- Use simple English.

- Mention three benefits of saving money.

- Give practical advice.

Observation

- The AI responds from the perspective of a financial advisor.
- The advice is practical and professional.
- The tone matches the assigned role.

Example Of Negative Based Prompting

Explain the importance of smartphones in education.
Requirements:
- Use simple English.
- Do not mention any brand names.
- Do not discuss social media.
- Limit the answer to 100 words.

Observation

- No smartphone brand names are mentioned.
- Social media is excluded.
- The response follows the requested length.

3. Constraint Specification & Iterative Refinement: Select an open-ended task (e.g., summarizing a technical article); issue a basic prompt; identify failures in length or format; refine the prompt by adding explicit constraints (word count, bullet format, etc.); document improvements over two refinement cycles

Aim

To understand how specifying constraints and refining prompts improves the quality, structure, and accuracy of AI-generated responses.

Objective

- Understand the concept of constraint specification.
- Learn iterative prompt refinement.
- Compare responses before and after adding constraints.
- Observe improvements in length, format, and content.

Task: Summarize a technical article on **Cloud Computing** through two refinement cycles.

Prompt

Summarize the following technical article.

Cloud Computing is the delivery of computing services such as servers, storage, databases, networking, software, and analytics over the Internet. It allows users to access computing resources on demand without maintaining physical infrastructure. Cloud computing offers scalability, flexibility, cost savings, and improved collaboration. It is widely used in education, healthcare, banking, e-commerce, and software development. However, challenges include data security, privacy concerns, vendor lock-in, and dependence on Internet connectivity.

Failure Analysis

- Response length is not controlled.
- Output is in paragraph format.
- Some important applications are omitted.
- No specific structure.

Now Refined Prompt

Summarize the following technical article.

Requirements:

- Use exactly 70 words.

- Present the answer in bullet points.

- Mention at least three application areas.

- Mention two advantages.

- Mention two challenges.

- Use simple English suitable for engineering students.

Article:

Cloud Computing is the delivery of computing services such as servers, storage, databases, networking, software, and analytics over the Internet. It allows users to access computing resources on demand without maintaining physical infrastructure. Cloud computing offers scalability, flexibility, cost savings, and improved collaboration. It is widely used in education, healthcare, banking, e-commerce, and software development. However, challenges include data security, privacy concerns, vendor lock-in, and dependence on Internet connectivity.

Improvements

- Follows the bullet-point format.
- Includes the requested application areas.
- Mentions the required advantages and challenges.
- Uses simple and structured language.
- Better organized and easier to read.

Exercise –3

1. **Structured Format Prompting: Instruct the model to output information as bullet lists and Markdown tables (e.g., "List three benefits of daily exercise in a Markdown table with columns 'Benefit' and 'Description.'"); verify the output matches the requested structure.**

Aim

To understand how structured format prompting enables AI models to generate responses in specific formats such as bullet lists and Markdown tables.

Objective

- Learn the concept of structured format prompting.
- Generate AI responses in bullet list and Markdown table formats.
- Verify whether the AI follows the requested output structure.
- Compare structured output with unstructured output.

Structured Format Prompting is a prompt engineering technique in which the user explicitly specifies the desired output format, such as:

- Bullet lists
- Numbered lists
- Markdown tables
- JSON
- XML

Providing formatting instructions helps the AI generate well-organized, readable, and consistent responses.

Part A: Bullet List Prompt

Prompt

List three benefits of daily exercise.

Requirements:

- Present the answer as a bullet list.

- Use simple English.

- Limit each point to one sentence.

Sample AI Response

- Improves physical health by strengthening muscles and the heart.
- Reduces stress and improves mental well-being.
- Increases energy levels and helps maintain a healthy weight.

Part B: Markdown Table Prompt

Prompt

List three benefits of daily exercise in a Markdown table with columns "Benefit" and "Description."

Requirements:

- Include exactly three rows.

- Use simple English.

Sample AI Response

Benefit	Description
Better Health	Regular exercise improves heart health and strengthens muscles.
Stress Reduction	Exercise reduces stress and improves mental well-being.
Weight Management	Daily exercise helps maintain a healthy body weight.

2. **JSON/YAML Generation: Provide a brief dataset description (e.g., three books with title, author, publication year) and prompt the model to produce valid JSON or YAML; use a parser to validate syntax and refine the prompt if errors occur.**

Aim

To generate structured data in JSON and YAML formats using prompt engineering, validate the generated syntax using a parser, and refine the prompt if necessary.

Objective

- Understand JSON and YAML generation using AI.
- Learn how to prompt AI to produce valid structured data.
- Validate the generated output using a JSON/YAML parser.
- Refine prompts to obtain syntactically correct output.

JSON (JavaScript Object Notation) and **YAML (YAML Ain't Markup Language)** are widely used data formats for storing and exchanging structured information.

Part A: Generate JSON

Prompt

Generate a valid JSON array for the following dataset.

Dataset:

1. Introduction to Artificial Intelligence, Stuart Russell, 2021

2. Deep Learning, Ian Goodfellow, 2016

3. Python Crash Course, Eric Matthes, 2023

Requirements:

- Use the keys "title", "author", and "publication_year".

- Return only valid JSON.

- Do not include explanations or extra text.

Part B: Generate YAML

Prompt

Generate a valid YAML document for the following dataset.

Dataset:

1. Introduction to Artificial Intelligence, Stuart Russell, 2021

2. Deep Learning, Ian Goodfellow, 2016

3. Python Crash Course, Eric Matthes, 2023

Requirements:

- Use the fields title, author, and publication_year.

- Return only YAML.

- Do not include explanations.

3. **Chain-of-Thought & Task Decomposition: Present a multi-step problem (e.g., a logic puzzle) and apply zero-shot CoT prompting (e.g., "Let's think step by step. Explain your reasoning before the final answer."); separately, decompose the problem into sequential sub-questions, collect partial answers, combine them, and compare accuracy against a direct-answer baseline.**

Aim : simple logic puzzle and a clear comparison between Direct Answer, Zero-Shot CoT, and Task Decomposition.

Direct Answer :Ask for final answer directly

Zero-Shot CoT :Ask the model to work through the problem step by step

Task Decomposition :Break the problem into smaller questions

Problem :

Three friends — **Rahul, Priya, and Kiran** — have different ages.

1. Rahul is older than Priya.
2. Kiran is older than Rahul.
3. Priya is 20 years old.
4. Rahul is 5 years older than Priya.

1. Direct Answer Prompt

The user directly asks the AI for the answer without asking it to explain the problem step by step.

Prompt :

Three friends — Rahul, Priya, and Kiran — have different ages.

1. Rahul is older than Priya.

2. Kiran is older than Rahul.

3. Priya is 20 years old.

4. Rahul is 5 years older than Priya.

Who is the oldest, and what are Rahul's and Priya's ages?

Give the final answer.

Predict Answer :

Arjun.

Observation:

The AI gives the correct answer, but the reasoning process is not requested.

2 . Zero-Shot Chain-of-Thought

Prompt:

Three friends — Rahul, Priya, and Kiran — have different ages.

1. Rahul is older than Priya.
2. Kiran is older than Rahul.
3. Priya is 20 years old.
4. Rahul is 5 years older than Priya.

Who is the oldest, and what are Rahul's and Priya's ages?

Let's think step by step and explain the reasoning before giving the final answer.

Sample AI Response:

Observation :

The model follows the relationships in the problem and arrives at the correct answer.

3 . Task Decomposition

Break the problem into smaller questions

Step 1

How old is Priya?

Answer:

Step 2

Rahul is 5 years older than Priya. How old is Rahul?

Answer:

Step 3

Kiran is older than Rahul. Who is the oldest?

Answer:

Step 4

Combine the answers. Who is the oldest, and what are Rahul's and Priya's ages

Exercise –4

1. Building a Simple LCEL Chain: Create a minimal LCEL script that accepts a fixed instruction (e.g., “Summarize this text: …”), passes it to an LLM, and prints the result; verify end-to-end execution.

Note:

LangChain is a framework that helps developers build applications using Large Language Models (LLMs) such as GPT, Gemini, and other AI models.

or

LangChain connects an LLM with prompts, data, tools, and other components to build AI applications.

Ans :

step-1: open https://openrouter.ai/

step-2: go to search choose : gpt-oss-safeguard-20b

step-3: then click the create api key

step-4: then click the new key

step-5: give name for that key

step-6: copy the key and paste in note pad then save

Cell 1 : install lang cahin package or module

```
!pip install -q langchain langchain-openai
```

Cell 2 : For load API into session

```
import os
from getpass import getpass
api_key = getpass("Enter your Open router API key:")
os.environ["OPENROUTER_API_KEY"] = api_key
print("API Key loaded successfully")
```

Cell3: Import lagchain

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(
    model="openai/gpt-oss-safeguard-20b",
    temperature=0,
    base_url="https://openrouter.ai/api/v1",
    api_key=os.environ["OPENROUTER_API_KEY"]
)

print("LLM Connected Successfully")
```

Cell4: Test the api is working or not

```
response = llm.invoke("Say Hello in one sentence.")

print(response.content)
```

Cell5 : Create llm cahin

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# Prompt
prompt = ChatPromptTemplate.from_template(
    "Summarize the following text in simple English:\n\n{text}"
)

# Output Parser
parser = StrOutputParser()

# LCEL Chain
chain = prompt | llm | parser

print("LCEL Chain Created Successfully")
```

Cell 6: Predicting summarized the text

```
text = """
Generative Artificial Intelligence is a modern technology that enables
computers to generate new content such as text, images, music, videos,
and computer programs. It learns patterns from huge amounts of data
and produces meaningful outputs according to user instructions.
"""

result = chain.invoke({"text": text})

print("Summary:")
print(result)
```

2. **Basic Data Indexing for RAG: Load a small collection of documents; split into uniform chunks (e.g., 200 tokens); generate embeddings for each chunk; store them in an in-memory vector store; inspect for consistency.**

In RAG, we do this:

Document

↓

Chunk

↓

Embedding

↓

Vector

↓

FAISS

↓

Find similar vectors

`=> faiss-cpu`, it is a Python package used for searching similar vectors (embeddings).

FAISS = Facebook AI Similarity Search

=> SentenceTransformer

This is a Python class from the `sentence-transformers` library.

Text

↓

SentenceTransformer

↓

Numbers (Embedding)

`"all-MiniLM-L6-v2"` it is a pre trained model

An **embedding is a way of representing text as numbers (a vector) so that a computer can compare the meaning of different texts.**

Solution:

Cell 1 :

```
!pip install -q sentence-transformers faiss-cpu
```

Cell 2:

```
from sentence_transformers import SentenceTransformer

import faiss

print("Sentence Transformers: OK")

print("FAISS: OK")
```

Cell 3:

```
documents = [

    "Artificial Intelligence is a branch of computer science that enables machines to perform tasks that normally require human intelligence.",


    "Machine Learning is a subset of Artificial Intelligence. It allows computers to learn patterns from data and make predictions.",


    "Generative AI can create new content such as text, images, audio, video, and computer code."

]


print("Number of documents:", len(documents))
```

Cell 4 :

```
chunks = []


for doc in documents:

    words = doc.split()
```

```
    for i in range(0, len(words), 50):
        chunk = " ".join(words[i:i+50])
        chunks.append(chunk)

print("Number of chunks:", len(chunks))

for i, chunk in enumerate(chunks):
    print(f"\nChunk {i+1}:")
    print(chunk)
```

Cell 5 :

```
model = SentenceTransformer("all-MiniLM-L6-v2")
embeddings = model.encode(chunks)
print("Embedding shape:", embeddings.shape)
```

Cell 6 :

```
import numpy as np
embeddings = np.array(embeddings).astype("float32")
dimension = embeddings.shape[1]
index = faiss.IndexFlatL2(dimension)
index.add(embeddings)
print("Number of vectors:", index.ntotal)
print("Vector dimension:", dimension)
```

Cell 7 :

```
query = "What is Machine Learning?"
query_embedding = model.encode([query]).astype("float32")
distances, indices = index.search(query_embedding, k=2)
print("Query:", query)
for i in indices[0]:
    print("\nRetrieved chunk:")
    print(chunks[i])
```

C. Constructing & Running a Basic RAG Chain: Build a pipeline that:

a. Receives a user query
b. Retrieves the top-k relevant chunks
c. Constructs a combined prompt with context + query
d. Send it to the LLM
e. Returns the answer

Test with sample queries and compare factual accuracy against a prompt without retrieval.

Solution

```
!pip install -q sentence-transformers faiss-cpu google-generativeai


import faiss
import numpy as np
from sentence_transformers import SentenceTransformer
from google.colab import userdata


embedding_model = SentenceTransformer("all-MiniLM-L6-v2")
print("Embedding model loaded successfully!")
api_key = userdata.get("GOOGLE_API_KEY")

print("API key loaded:", api_key is not None)


from google import genai
client = genai.Client(api_key=api_key)
print("Gemini client configured successfully!")
documents = [
        """
    Deep Learning is a type of Machine Learning that uses neural
networks
    with multiple layers. Deep Learning is widely used in image
recognition,
    speech recognition and natural language processing.
    """,
    """
    Generative AI is a type of artificial intelligence that can create
new
    content such as text, images, audio, video and computer code.
    Large Language Models are widely used in Generative AI
applications.
    """,
```

```
    """
    Large Language Models, or LLMs, are AI models trained on large
amounts
    of text data. They learn language patterns and can generate text,
    answer questions, summarize information and perform many language
tasks.
    """
]
print("Number of documents:", len(documents))
chunks = []
for document in documents:
    chunks.append(document.strip())

print("Number of chunks:", len(chunks))
for i, chunk in enumerate(chunks):
    print(f"\n--- Chunk {i+1} ---")
    print(chunk)
chunk_embeddings = embedding_model.encode(chunks)
print("Embedding shape:", chunk_embeddings.shape)
chunk_embeddings = np.array(chunk_embeddings).astype("float32")
dimension = chunk_embeddings.shape[1]
print("Embedding dimension:", dimension)
index = faiss.IndexFlatL2(dimension)
index.add(chunk_embeddings)
print("Number of vectors stored:", index.ntotal)
query = "What is Machine Learning?"
print("User Query:", query)
query_embedding = embedding_model.encode([query])
query_embedding = np.array(query_embedding).astype("float32")
print("Query embedding shape:", query_embedding.shape)
k = 2
distances, indices = index.search(query_embedding, k)
print("Retrieved chunk indices:", indices)
print("Distances:", distances)
for i, idx in enumerate(indices[0]):
    print(f"\n--- Retrieved Chunk {i+1} ---")
    print(chunks[idx])
```

Exercise –5

A. To build a simple LLM agent that registers a calculator tool, uses prompts to decide when to invoke the tool, and tests queries that require tool execution using LangChain.

- Import the required LangChain libraries.
- Define a calculator function to perform arithmetic operations.
- Register the calculator function as a tool using the `@tool` decorator.
- Create an LLM agent and provide the calculator tool.
- Define a system prompt instructing the agent to use the calculator when calculations are required.
- Provide test queries to the agent.
- Execute the agent and display the responses.
- Verify whether the calculator tool is invoked for mathematical queries.

Solution :

```
from langchain.agents import create_agent

from langchain.tools import tool

# Register the calculator tool

@tool

def calculator(a: float, b: float, operation: str) -> str:

    """Perform arithmetic operations:

    add, subtract, multiply, or divide."""


    if operation == "add":

        result = a + b

    elif operation == "subtract":

        result = a - b

    elif operation == "multiply":

        result = a * b

    elif operation == "divide":
```

```
        if b == 0:
            return "Cannot divide by zero"
        result = a / b
    else:
        return "Invalid operation"
    print("Calculator tool executed")
    return str(result)
# Create the LLM agent
agent = create_agent(
    model="openai:gpt-4.1-mini",
    tools=[calculator],
    system_prompt="""
    You are a helpful AI assistant.
    If a question requires mathematical calculation,
    use the calculator tool.
    Do not guess calculation results.
    Answer other questions normally.
    """
)
# Test the agent
queries = [
    "What is 25 multiplied by 4?",
    "Add 100 and 250.",
    "What is an LLM?"
]
for query in queries:
    print("\nUser:", query)
```

```
response = agent.invoke({
    "messages": [
        {"role": "user", "content": query}
    ]
})
print("Agent:", response["messages"][-1].content)
```